Faculdade de Tecnologia de Sorocaba
Tecnologia em Análise e Desenvolvimento de Sistemas

NICOLLY LEMOS DA SILVA

DOCUMENTAÇÃO DE SOFTWARE:
SISTEMA DE AGENDAMENTO ONLINE

SOROCABA/SP
2026

Repositório: https://github.com/nicollylemos/Agendamento

1. Objetivo

Oferecer a pequenos negócios de serviços, como salões e barbearias, um agendamento online próprio. O cliente escolhe serviço, profissional e horário pelo site e garante a vaga pagando um sinal via Pix. A equipe gerencia agenda, estoque, fidelidade e relatórios num painel único.

2. Problema

Muitos desses negócios ainda agendam pelo WhatsApp ou no caderno, o que gera troca excessiva de mensagens, clientes que faltam sem avisar e horários marcados em dobro. Também falta controle de estoque, de custos e de faturamento, e as plataformas prontas cobram por profissional e raramente reúnem tudo isso num só lugar.

3. Escopo

O sistema inclui site público personalizável, agendamento online sem cadastro, sinal via Pix com confirmação automática, painel da equipe, estoque, programa de fidelidade, relatórios e lembretes por e-mail. Ficaram fora desta versão o suporte a várias empresas, o app nativo, os lembretes por WhatsApp e a emissão de nota fiscal.

4. Requisitos funcionais

Área	Códigos	Resumo
Acesso	RF01–RF05	Login com 2FA opcional, convite da equipe, recuperação de senha e acesso do cliente por código
Site	RF06–RF11	Dados da empresa, personalização do site e prévia antes de publicar
Serviços	RF12–RF15	Cadastro de serviços e profissionais, ficha técnica e cálculo de margem
Horários	RF16–RF20	Jornadas, folgas, bloqueios e cálculo dos horários livres
Agendamento	RF21–RF27	Agendar, remarcar, cancelar, controlar status e enviar lembretes
Sinal e Pix	RF28–RF37	Cobrança do sinal, QR Code, confirmação automática e expiração
Estoque	RF38–RF42	Produtos, movimentações, baixa automática e alertas
Fidelidade	RF43–RF46	Pontos ou selos, recompensas e extrato do cliente
Relatórios	RF47–RF55	Faturamento, ocupação, comissões e exportação em CSV e PDF

5. Requisitos não funcionais

Código	Requisito
RNF01	O banco impede dois agendamentos no mesmo horário para o mesmo profissional
RNF02	Senhas com argon2id e checagem contra senhas vazadas
RNF03	Permissões por perfil (Admin, Gerente, Funcionário) verificadas no servidor
RNF04	Dados sensíveis criptografados e limite de tentativas de login
RNF05	Valor do sinal calculado no servidor e pagamento conferido no gateway
RNF06	Interface responsiva e em português
RNF07	Testes automáticos a cada envio ao GitHub

6. Tecnologias utilizadas

Camada	Tecnologias
Backend	Node.js 22, Express 5, TypeScript, Prisma
Banco e filas	PostgreSQL 16, Redis 7 e BullMQ
Frontend	React 19, Vite e Tailwind CSS
Pagamentos	Mercado Pago (Pix)
Testes e CI	Vitest e GitHub Actions

7. Arquitetura e componentes

O sistema é cliente-servidor. Um frontend em React serve o site e o painel e consome uma API REST em Express, que grava os dados no PostgreSQL e usa o Redis para limitar requisições e rodar tarefas em segundo plano, como expirar sinais não pagos e enviar lembretes.

O projeto é um monorepo com três partes: apps/api, apps/web e packages/shared. Na API, cada domínio (auth, agendamentos, pagamentos, estoque, fidelidade, relatórios) tem sua própria pasta em src/modules. No front, as telas ficam em src/pages, separadas em site público, painel do admin e área do profissional.

No agendamento, o horário fica reservado assim que o cliente confirma, e a API gera o Pix. Quando o pagamento cai, o Mercado Pago avisa por webhook e o agendamento é confirmado. Se o prazo passar, o horário é liberado.

8. Banco de dados

O banco é PostgreSQL, com 32 tabelas modeladas no Prisma e versionadas por migrations. As principais são usuarios, funcionarios, servicos, clientes, agendamentos, pagamentos, produtos e transacoes_fidelidade. Uma constraint no próprio banco bloqueia agendamentos sobrepostos, e triggers impedem que o histórico de estoque e de pontos seja alterado.

9. APIs

A API é REST, responde em JSON e usa o prefixo /api/v1. As rotas se dividem em públicas (site e agendamento), autenticação, administrativas (/admin), da equipe (/staff) e o webhook de pagamento. A lista completa está em docs/API.md.

10. Variáveis de ambiente

Ficam em apps/api/.env, a partir do modelo .env.example. As principais são DATABASE_URL e REDIS_URL (conexões), JWT_PRIVATE_KEY, JWT_PUBLIC_KEY e ENCRYPTION_KEY (chaves geradas com npm run gerar-chaves) e APP_URL (endereço do front). A API não inicia se faltar alguma.

11. Testes e deploy

São cerca de 190 testes com Vitest, entre unitários e de integração, executados com npm test e também pelo GitHub Actions a cada envio. O deploy previsto é numa VPS Linux com Nginx e pm2, e o passo a passo está no README do repositório.